

[New issue](#)

cargo zigbuild doesn't support static glibc builds with an explicit `--target` #231

[Open](#)

polarathene opened on Mar 7, 2024



cargo zigbuild is not compatible with `RUSTFLAGS="-C target-feature=+crt-static"`, aka static builds. (**EDIT:** At least when an explicit `--target` is configured)

Reproduction

```
# Reproduction environment:
$ docker run --rm -it fedora:40 bash
$ dnf install -y cargo
$ cargo init /tmp/example && cd /tmp/example

# Build will fail:
$ RUSTFLAGS="-C target-feature=+crt-static" cargo build --release --target x86_64-unknown
  Compiling example v0.1.0 (/tmp/example)
error: linking with `cc` failed: exit status: 1
# ...
= note: /usr/bin/ld: cannot find -lm: No such file or directory
       /usr/bin/ld: cannot find -lc: No such file or directory
       /usr/bin/ld: cannot find -lc: No such file or directory
       collect2: error: ld returned 1 exit status

# Fix:
$ dnf install -y glibc-static
$ RUSTFLAGS="-C target-feature=+crt-static" cargo build --release --target x86_64-unknown
  Compiling example v0.1.0 (/tmp/example)
  Finished release [optimized] target(s) in 0.11s
```

Now with cargo zigbuild:

```
# Install cargo-zigbuild + zig:
$ cargo install cargo-zigbuild
$ dnf install -y pip
$ pip install ziglang

# Build with `cargo zigbuild`:
$ RUSTFLAGS="-C target-feature=+crt-static" cargo zigbuild --release --target x86_64-unknown
```

```

    Compiling example v0.1.0 (/tmp/example)
error: linking with `/root/.cache/cargo-zigbuild/0.18.3/zigcc-x86_64-unknown-linux-gnu.sh
# ...
= note: error: unable to find Static system library 'gcc_eh' using strategy 'no_fallbac
/tmp/example/target/x86_64-unknown-linux-gnu/release/deps/libgcc_eh.a
/tmp/example/target/release/deps/libgcc_eh.a
/usr/lib/rustlib/x86_64-unknown-linux-gnu/lib/libgcc_eh.a
/usr/lib/rustlib/x86_64-unknown-linux-gnu/lib/libgcc_eh.a
error: unable to find Static system library 'gcc' using strategy 'no_fallback'.
/tmp/example/target/x86_64-unknown-linux-gnu/release/deps/libgcc.a
/tmp/example/target/release/deps/libgcc.a
/usr/lib/rustlib/x86_64-unknown-linux-gnu/lib/libgcc.a
/usr/lib/rustlib/x86_64-unknown-linux-gnu/lib/libgcc.a

# Build again but verify without static works:
cargo zigbuild --target x86_64-unknown-linux-gnu --release
    Compiling example v0.1.0 (/tmp/example)
    Finished release [optimized] target(s) in 2.28s

# With custom glibc version support:
cargo zigbuild --target x86_64-unknown-linux-gnu.2.32 --release
    Compiling example v0.1.0 (/tmp/example)
    Finished release [optimized] target(s) in 2.18s

# Without custom glibc version support, avoiding the `--target` option works:
RUSTFLAGS="-C target-feature=+crt-static" cargo zigbuild --release
    Compiling example v0.1.0 (/tmp/example)
    Finished release [optimized] target(s) in 0.15s

```

It seems that `cargo-zigbuild` (~~or zig itself?~~) cannot support statically linked builds?

If you use the official Rust image (*Debian via `rust:latest`*), you can perform the static build without installing any extra package like `glibc-static`, but you'll still get the same error when trying with `cargo zigbuild`.

Insights

There is a related [Github Discussion about this problem](#) from Sep 2023, but no solution.

- I've raised an issue here for better visibility and to demonstrate it affects the basic "Hello, world!" example from `cargo init`.
- The author of that linked discussion appears to have [raised their issue upstream to the zig project](#), but no response yet.

Given those last findings that it works without `--target` or without the static flag, it seems this is not an upstream zig issue, but related to Cargo?

- `--target` is required usually for a static build if any proc macro is involved, otherwise you'll get build errors. `--target` will avoid using static on the build scripts IIRC, while still using it for your build target during compilation.
- `gcc_eh` AFAIK is related to panic support, while Clang offers `libunwind` as an alternative, from what I've read you can only have a single implementation and that gets complicated if you have

deps that are C/C++ related.

- I can't recall if that's relevant to static builds, other than devs stating only one of these should be used which complicates using `libunwind`.
- I also recall that this gotcha affected static builds that don't actually need it (*`no_std` with `panic = "abort"` I think is an example*), but `rustc` hard-codes this link requirement anyway? 🤖 (similarly for the `libgcc.a` requirement)

My interest was in reproducing [this example](#), which depends on `glibc 2.32` specifically.

- I can build for `glibc 2.32` without `cargo-zigbuild` via Fedora 33 and using it's `glibc-static` package which matches that version.
- I figured this would have been a good reason to try `cargo zigbuild` but was surprised that it didn't work, and that there is barely any discussion about the failure 😬



🔗 **polarathene** mentioned this on Mar 7, 2024

🔗 [unable to find Static system library 'gcc eh' ziglang/zig#17268](#)



messense on Mar 7, 2024

Member ...

Given those last findings that it works without `--target` or without the static flag, it seems this is not an upstream zig issue, but related to Cargo?

Not really, if you don't pass `--target`, zig won't be used, it's effectively running `cargo build`.



messense on Mar 7, 2024 · edited by messense

Edits ▼

Member ...

t.c :

```
#include <stdio.h>

int main(void) {
    printf("hello");
    return 0;
}
```



build with `zig cc -static` ends up with a non-static executable

```
$ python3 -m ziglang cc -static -o t t.c
```

```
$ ldd t
```

```
linux-vdso.so.1 (0x00007ffc22dfb000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f72e7e00000)
/lib64/ld-linux-x86-64.so.2 (0x00007f72e8186000)
```

Seems that `zig cc` isn't able to produce a fully static executable that uses glibc? See also [ziglang/zig#4986](https://github.com/ziglang/zig/issues/4986)



haohaolee on Mar 7, 2024

Contributor



Building the static binary with glibc is not recommended actually, because glibc is not designed this way. See this reference <https://stackoverflow.com/questions/57476533/why-is-statically-linking-glibc-discouraged>

If static binaries are wanted, the goto approach is `x86_64-unknown-linux-musl`



2



polarathene on Mar 7, 2024

Contributor

Author



Not really, if you don't pass `--target`, zig won't be used, it's effectively running `cargo build`.

Probably worth mentioning that ([README section doesn't clarify that gotcha](#)), it also seems to be the case if I use `RUSTFLAGS="-C linker=cc"`?

I noticed that I could specify an invalid glibc version in the target too and that still builds successfully, but a little more surprising was that building with dynamic link to version `2.32` did not reproduce the expected failure

```
# In Fedora 33 (glibc 2.32):
$ cargo build --release --target x86_64-unknown-linux-gnu
# In Fedora 32 (glibc 2.31):
$ ./example
./example: /lib64/libc.so.6: version `GLIBC_2.32' not found (required by ./example)

# Build on Fedora 33 (with cargo-zigbuild):
$ cargo zigbuild --release --target x86_64-unknown-linux-gnu.2.32
# Run on Fedora 32:
$ ./example
Hello, world!

# Build on Fedora 33:
$ cargo zigbuild --release --target x86_64-unknown-linux-gnu.2.33
# Run on Fedora 32:
$ ./example
./example: /lib64/libc.so.6: version `GLIBC_2.33' not found (required by ./example)

# Fedora 32:
$ ldd --version
ldd (GNU libc) 2.31
```



```
$ dnf info glibc
Installed Packages
Name      : glibc
Version   : 2.31
```

- Not sure why `cargo zigbuild` doesn't fail with 2.32 requirement when only 2.31 is available, yet does this for 2.33? 🤔
- For an invalid version if invoking `zig cc` directly with an equivalent hello world C program I got this warning:

```
LLD Link... warning: zig cannot build new glibc version 2.320.0; providing
instead 2.34.0
```

UPDATE: On Fedora 33, I was able to build without the linking error by including `-L /usr/lib/gcc/x86_64-redhat-linux/10`, but similar to `-C linker=cc` this seems to result in always using glibc 2.32 from the system?

- An explicit `--target` doesn't seem to make any difference in adding the `.2.31 / 2.33` suffix to the target.
- `ldd` claims static linked.
- `file` claims dynamic linked with interpreter (`/lib64/ld-linux-x86-64.so.2`).

Fedora 33 (creates a dynamically linked binary despite static request):

```
$ RUSTFLAGS="-C target-feature=+crt-static -L /usr/lib/gcc/x86_64-redhat-linux/10" cargo
```



```
$ ldd target/x86_64-unknown-linux-gnu/release/example
linux-vdso.so.1 (0x00007ffe6af58000)
libpthread.so.0 => /lib64/libpthread.so.0 (0x00007fcd227cf000)
libc.so.6 => /lib64/libc.so.6 (0x00007fcd22604000)
/lib64/ld-linux-x86-64.so.2 (0x00007fcd227fa000)
libutil.so.1 => /lib64/libutil.so.1 (0x00007fcd225ff000)
```

Without `cargo-zigbuild`, here is a static build on Fedora 33. `ldd` will state it is statically linked, `file` will say it is dynamic still unless adding `-C relocation-model=static`

Fedora 33:

```
RUSTFLAGS="-C target-feature=+crt-static" cargo build --release --target x86_64-unknown-l
Compiling libc v0.2.153
Compiling example v0.1.0 (/tmp/example)
Finished release [optimized] target(s) in 0.44s
```



Fedora 32 (this is with the code `for` triggering a specific failure when linking glibc 2

```
$ ./example
./example: dl-call-libc-early-init.c:37: _dl_call_libc_early_init: Assertion `sym != NULL'
# If it were dynamically linked, it'd fail with this error instead due to the incompatibl
./example: /lib64/libc.so.6: version `GLIBC_2.32' not found (required by ./example)
```

Fedora 40 (glibc 2.39) that same binary segfaults, but would otherwise work fine `if` dyn

```
$ ./example
Segmentation fault
```

Fedora 33 binary appears static?:

```
$ ldd target/x86_64-unknown-linux-gnu/release/example
```

not a dynamic executable

```
$ file target/x86_64-unknown-linux-gnu/release/example
target/x86_64-unknown-linux-gnu/release/example: ELF 64-bit LSB executable, x86-64, versi

# `dl_open` usage from glibc makes it secretly dynamic still:
$ nm -an target/x86_64-unknown-linux-gnu/release/example | grep dl_open
000000000049b6e0 t .annobin_dl_open.c
000000000049baa2 t .annobin__dl_open.start
000000000049bab0 T _dl_open
000000000049bd04 t .annobin__dl_open.end
000000000049be42 t .annobin_dl_open_worker.start
000000000049be50 t dl_open_worker
000000000049c80c t .annobin_dl_open.c_end
000000000049c80c t .annobin_dl_open_worker.end
00000000004a12ef t .annobin__libc_register_dl_open_hook.start
00000000004a12f0 T __libc_register_dl_open_hook
00000000004a1333 t .annobin__libc_register_dl_open_hook.end
0000000000543740 d _dl_open_hook
```

Seems that `zig cc` isn't able to produce a fully static executable that uses glibc? See also [ziglang/zig#4986](https://github.com/ziglang/zig/issues/4986)

I've [commented there](#) trying to build static hello world by the advice to provide the extra `.a` files which seems to be related to the gcc files failing here, but even the `hello.c` example is still missing something.

It now claims to be statically linked via `ldd`, but segfaults, presumably due to the `dl_open` that is still present? (*unlike my snippets above, the segfault is occurring on the same build host with `zig cc`*)

Building the static binary with glibc is not recommended actually, because glibc is not designed this way. See this reference

[I am aware of this](#), I wanted to reproduce some of the issues related to it for documenting this concern (*in particular, [this one](#)*). As shown above I can't reproduce when using Zig to specify the glibc (*even when using dynamic linking, the behaviour is unexpected?*)

For some programs like a basic hello world it should be fine to static link with glibc AFAIK, while the NSS caveat kind of applies to a static build with musl too?

If static binaries are wanted, the goto approach is `x86_64-unknown-linux-musl`

Yeah I use that, but that has some gotchas too:

- If you're building with external dependency like `openssl`, it is not as friendly DX to create static build with musl target unless using Alpine or similar? Many reports still occur from users not having the right packages installed to support that, or they workaround it by compiling openssl with musl from a gnu build host.

- Default memory allocator is fairly poor at multi-thread workloads. Need to actively change it, which is not always convenient if compiling someone else's project. Can technically run something like mimalloc via `LD_PATH` as a workaround (*in `rust:alpine` this improved build speed by over 2x for me when invoking `cargo build`*).
- There is talk about this target changing away from the default static link behaviour, which is not only `-C target-feature=+crt-static` but also `-C link-self-contained=yes`. In the meantime it does confuse some users depending upon how they install rust toolchain within Alpine or other musl distros that default to dynamic linking already.
- It's had its own DNS issues in the past IIRC?



messense on Mar 8, 2024

Member



I really appreciate the detailed write-up.

FYI, static linking glibc has never been the goal of this project so it has never been tested. But I'm open to merge PRs that enable this if you really want it and willing to invest time to coding and testing.



1



polarathene on Mar 8, 2024 · edited by polarathene

Edits ▾

Contributor

Author



But I'm open to merge PRs that enable this if you really want it and willing to invest time to coding and testing.

I don't know how to successfully use `cargo-zigbuild` for a static glibc build like I was able to do without it on the Fedora 33 container to target `2.32`.

As you helped point out, that seems to be an upstream issue with Zig's static linking support, which is possibly complicated further when static linking libc since it's doing its own thing for the glibc version feature support it has?

Until there is an answer there on how to statically compile `hello.c` without it segfaulting (*especially on the same build host*), I don't think there's anything `cargo-zigbuild` can do. Providing the `-L` location to satisfy the gcc errors didn't really accomplish anything since the static request was ignored and we got an obviously dynamically linked executable anyway 🤔

For now, I think it's better to just point out that `cargo-zigbuild` does not support static linking in the README. It can reference this issue for more context to the reader, that way it keeps the README mention simple.

EDIT: [PR ready](#), hope that covers it well 🍀



  **polarathene** mentioned this on Mar 8, 2024

 [zig cc: detect -static and treat -l options as static library names ziglang/zig#4986](#)

 [docs: Add caveats for glibc version targeting feature #232](#)



haohaolee on Mar 8, 2024 · edited by haohaolee

Edits ▾

Contributor



One thing that I am wondering is why you insist on running a binary built with glibc 2.32 on a system with glibc 2.31 with static linking.

glibc only guarantees backwards compatibility - binaries built with lower glibc are supposed to run correctly on higher glibc, but not vice versa.

Static linking is not a panacea, it does not solve compatibility issues.

If you prefer glibc, the very traditional approach is building your app against a low version of glibc (rust can support as low as 2.17) with dynamically linking, and everything should be fine.

Another approach is redistributing your own glibc of whatever new version you like with your app, which should work also but needs more effort.



haohaolee on Mar 8, 2024

Contributor



If you're building with external dependency like openssl, it is not as friendly DX to create static build with musl target unless using Alpine or similar? Many reports still occur from users not having the right packages installed to support that, or they workaround it by compiling openssl with musl from a gnu build host.

Not that hard with zigbuild?

Just tried the following in fedora container with this [example](#):

```
[root@9ea04f2aa461 hello-tls]# cargo zigbuild --release --target x86_64-unknown-linux-musl
Compiling openssl-sys v0.9.96
Compiling tokio v1.27.0
Compiling futures-core v0.3.28
Compiling mio v0.8.11
Compiling tokio-macros v2.0.0
Compiling socket2 v0.4.9
Compiling num_cpus v1.15.0
Compiling itoa v1.0.6
Compiling futures-task v0.3.28
....
Compiling hello-tls v0.1.0 (/root/cargo-zigbuild/tests/hello-tls)
Finished release [optimized] target(s) in 6m 23s
```

```
[root@9ea04f2aa461 hello-tls]# file target/x86_64-unknown-linux-musl/release/hello-tls
target/x86_64-unknown-linux-musl/release/hello-tls: ELF 64-bit LSB executable, x86-64, ve
[root@9ea04f2aa461 hello-tls]# ./target/x86_64-unknown-linux-musl/release/hello-tls
Response status 200 OK
```




 polarathene on Mar 9, 2024

Contributor

Author

...

One thing that I am wondering is why you insist on running a binary built with glibc 2.32 on a system with glibc 2.31 with static linking.

glibc only guarantees backwards compatibility - binaries built with lower glibc are supposed to run correctly on higher glibc, but not vice versa.

I did not expect it to work, I wanted an example of static linking with a `-gnu` target that demonstrates that it breaks. That is one I came across which is known to be specific to glibc 2.32.

On glibc 2.31 or earlier you'd get that error I showed AFAIK:

```
./example: dl-call-libc-early-init.c:37: _dl_call_libc_early_init: Assertion 'sym != NULL' failed.`
```

While on newer glibc it appears to segfault (*which it won't if dynamically linked of course*), which is why it's specific to glibc 2.32 unlike others which AFAIK may still be forward compatible with glibc when static linking (*there is a subset that is IIRC and it improves with newer releases*).

In fact I think from 2.33, this method was changed to be compatible with static build so it shouldn't segfault when static linking to 2.33+ 👍

This also helps highlight the difference vs dynamic linking glibc where you'd otherwise expect a different error emitted when not compatible:

```
./example: /lib64/libc.so.6: version 'GLIBC_2.32' not found (required by ./example)`
```

However, I want to draw attention to Zig dynamic linking not emitting that error and working on glibc 2.31 instead. Yet if I target 2.33 instead of 2.32 it would emit the equivalent error as shown in the quote as you'd expect. So why didn't that happen for 2.32? 🤔

So to reiterate, I wanted an example of static linking glibc where breakage occurs. You don't get that with hello world, you need to call something that'd trigger the `dl_open` AFAIK? So this helps demonstrate the problem when I talk about static linking concerns with glibc / `-gnu` targets to others :)

Static linking is not a panacea, it does not solve compatibility issues.

It's about a portable build.

Some users will build with a `-gnu` target statically linked, look at the `ldd` output and trust that it's static since nothing appears dynamically linked, they may not be aware of the `dl_open` that can be used at runtime though and would fail.

For those that wonder about it and may reason that it seems fine for them instead of using a `-musl` target, they may advise to just static link `-gnu` target as often the advice at least with Rust discussions doesn't always touch on specifics or any reproducible example, some only mention NSS concerns if your program would call that.

I've seen this misunderstanding quite a few times and even personally been curious about how to reproduce a static glibc incompatibility for some time and I only recently got around to investigating how to.

If you prefer glibc, the very traditional approach is building your app against a low version of glibc (rust can support as low as 2.17) with dynamically linking, and everything should be fine.

Yes I'm aware of this advice 👍

I was interested in the static build context, since dynamic linking glibc would not be as portable as the musl static build. Musl has it's own gotchas to accommodate, I just needed to properly reproduce the big gotcha for glibc static.

Prior to Rust 1.72 there was a notable issue where static builds still accidentally dynamically linked even with musl, but those now surface at build time thankfully, whereas the glibc concern is more subtle and I imagine would still surprise some users not as familiar as yourself as to why it breaks :)

Another approach is redistributing your own glibc of whatever new version you like with your app, which should work also but needs more effort.

Sure, but the focus here was on a single distributable binary. For myself that's via GH releases and Docker.

I am not too experienced with dynamic linking libs that way, does relative paths work? Perhaps other gotchas? (I know with `ld -run` command it'd replace my `LD_PRELOAD` for `mimalloc` when running `cargo build` to speed up a long musl build by over 2x)



 polarathene on Mar 9, 2024

Contributor

Author

...

openssl building `-gnu` + `-musl` targets same host (*not specific to cargo zigbuild*)

EDIT: This was a misunderstanding of mine with the `vendored` feature for the `openssl` crate.

- I recalled it as requiring `-L` to reference your own build of openssl, not that it [built from source](#) 😓 (it seems I mixed that up with what I saw someone else try [here](#) with their own build using `OPENSSL_DIR`)
- While the `openssl` crate supports building from source via the `vendored` feature, I'm not sure [if that's always the case for crates with external deps](#)?

- I remember building a third-party project where I used `OPENSSL_NO_VENDOR` to opt-out as builds were failing with error messages that weren't too helpful.
 - Instead I used OpenSSL packages from Alpine and Debian to build the `-musl` + `-gnu` targets, but that did not seem to support building for both targets via a single image.
 - I was building with my own image since it seemed much simpler than what Vector was doing ([this](#) and [this](#) are musl specific, but the `cross` musl base image in that `Dockerfile` is actually Ubuntu_).
- Now that that mixup of mine is cleared up:
 - Building `openssl` works with `rust:latest` (Debian) and `-musl` as well (if I use `musl-tools` package, which provides `musl-gcc` in addition to `musl-dev`).
 - I can't seem to build for `-gnu` target from Alpine, but that's the case even with a basic "hello world" (`cargo init`) without the `openssl` crate 🤔

Original response follows.

Not that hard with zigbuild?

You are using [vendored](#) ([request:Cargo.toml](#) => [native-tls docs vendored feature](#) => enables [vendored feature of openssl](#) crate). I was referring to not using that, I'm not sure if all crates with external deps support building from source as a fallback but I recall running into build issues a few times in the past with crates, sometimes it's just the failures did not have very helpful error messages.

```
# Reproduction environment:
docker run --rm -it fedora:40 bash
dnf install -y gcc pip rustup
rustup-init -y && source "$HOME/.cargo/env"
rustup target add x86_64-unknown-linux-musl

# Install cargo-zigbuild + zig:
pip install ziglang
cargo install cargo-zigbuild

# Prepare project
cargo init /tmp/example && cd /tmp/example
cargo add openssl --features vendored
echo 'fn main() { println!("OpenSSL version is: {}", openssl::version::version()); }' > s

# `cargo build` / `cargo zigbuild` doesn't matter will fail:
cargo build --release --target x86_64-unknown-linux-musl
```



► Error output

You can install `openssl-dev` package with DNF to build successfully (if you remove the `vendored` feature), but this will only work for the `-gnu` target not `-musl`. It will of course fail with `-gnu` if you try to build with static, since Fedora does no longer offers a package variant for static openssl (Alpine does).

Thus you'd need to build OpenSSL as a static lib and point to it for the `vendored` feature to use?

EDIT: This needed `dnf install -y perl musl-gcc` to successfully build from source.

The `-gnu` target can do a static build with `perl + glibc-static` packages, but contains a `_dl_open` unlike the `-musl` target (*that may not be too relevant, `_dl_open` isn't present if building with dynamically linking and I think the symbol is introduced for static glibc regardless, even when nothing would actually call it at runtime?*).

Just tried the following in fedora container with this [example](#):

Just to confirm, your reference project isn't any different to above reproduction example:

```
# Extra commands for the reproduction reference, fails for same reason as above example
dnf install -y git
git clone https://github.com/rust-cross/cargo-zigbuild /tmp/cz && cd /tmp/cz/tests/hello-
cargo zigbuild --release --target x86_64-unknown-linux-musl
```

~~You only mention successfully building that in a Fedora container, but clearly there is some context missing?~~ (**EDIT:** Required `perl` for the `openssl-src` crate to build, while building for `musl` target needed `musl-gcc`)

cargo build vs cargo zigbuild linking difference when not using openssl/vendored

With my `/tmp/example` reproduction, once you add the `openssl-devel` package `cargo build` is successful, but `cargo zigbuild` fails for some reason 🤖

```
# No problems:
$ cargo build --release --target x86_64-unknown-linux-gnu
  Compiling foreign-types-shared v0.1.1
  Compiling bitflags v2.4.2
  Compiling once_cell v1.19.0
  Compiling cfg-if v1.0.0
  Compiling openssl-sys v0.9.101
  Compiling libc v0.2.153
  Compiling foreign-types v0.3.2
  Compiling openssl v0.10.64
  Compiling example v0.1.0 (/tmp/example)
  Finished release [optimized] target(s) in 2.59s

# Problems:
$ cargo zigbuild --release --target x86_64-unknown-linux-gnu
  Compiling foreign-types-shared v0.1.1
  Compiling once_cell v1.19.0
  Compiling openssl-sys v0.9.101
  Compiling cfg-if v1.0.0
  Compiling bitflags v2.4.2
  Compiling libc v0.2.153
  Compiling foreign-types v0.3.2
  Compiling openssl v0.10.64
  Compiling example v0.1.0 (/tmp/example)
```

```
error: linking with `/root/.cache/cargo-zigbuild/0.18.3/zigcc-x86_64-unknown-linux-gnu.sh
```

```
# ...
```

```
= note: error: unable to find Dynamic system library 'ssl' using strategy 'no_fallback'
/tmp/example/target/x86_64-unknown-linux-gnu/release/deps/libssl.so
/tmp/example/target/release/deps/libssl.so
/root/.rustup/toolchains/stable-x86_64-unknown-linux-gnu/lib/rustlib/x86_64-u
/root/.rustup/toolchains/stable-x86_64-unknown-linux-gnu/lib/rustlib/x86_64-u
error: unable to find Dynamic system library 'crypto' using strategy 'no_fallba
/tmp/example/target/x86_64-unknown-linux-gnu/release/deps/libcrypto.so
/tmp/example/target/release/deps/libcrypto.so
/root/.rustup/toolchains/stable-x86_64-unknown-linux-gnu/lib/rustlib/x86_64-u
/root/.rustup/toolchains/stable-x86_64-unknown-linux-gnu/lib/rustlib/x86_64-u
```

Which is a little odd since those are available, `cargo build` had no problem finding them. `dnf` provides `*libcrypto.so` `*libssl.so` shows that they're part of the `openssl-devel` package.

```
$ dnf repoquery -l openssl-devel | grep -E 'lib(crypto|ssl).so'
```



```
/usr/lib/libcrypto.so
/usr/lib/libssl.so
/usr/lib64/libcrypto.so
/usr/lib64/libssl.so
```

```
# Builds successfully only with `-L /usr/lib64`, not `-L /usr/lib` (despite including thi
$ RUSTFLAGS="-L /usr/lib64" cargo zigbuild --release --target x86_64-unknown-linux-gnu
```

```
$ target/x86_64-unknown-linux-gnu/release/example
OpenSSL version is: OpenSSL 3.2.1 30 Jan 2024
```

So `-L` can be used to provide a hint there, but as mentioned that's presently not compatible with `-C target-feature=+crt-static` (when I tried to successfully get a static `glibc` build for "hello world").

- Instead of failing, you get a dynamically linked build.. which was a little misleading.
- The `glibc` version targeting feature still works, so it's still going through `zig cc`, just ignoring that static build request.



haohaolee on Mar 10, 2024 · edited by haohaolee

Edits ▾

Contributor



Glad you have figured out the `openssl/vendored` issue. I forgot to mention `perl` is needed in the Fedora container, sorry for the inconvenience, but `musl-gcc` is not needed when using `zigbuild`. btw, you can try `cargo zigbuild --target aarch64-unknown-linux-musl` to get an arm `openssl` static binary right away. (`musl-gcc` cannot do this)

Therefore `zigbuild` is really a cross compiling tool, it does not help a lot when targeting the same host, especially when you want to explore static linking with `glibc`.

Most linker errors you showed above are because `zig` is used as a linker but it doesn't know more about the library search path on the host than the system linker `ld`. So you must supply the hint for it to link successfully.

Finally, `zig` is not designed to statically link to `glibc` (IMHO), it has its [own approach](#) to deal with `glibc` symbols and does not rely on the system `glibc` at all, that's why we can run `zigbuild` on even a non-linux system to get a linux binary.

I can't seem to build for `-gnu` target from Alpine, but that's the case even with a basic "hello world" (cargo init) without the `openssl` crate 🤔

Why? `zigbuild` should work for `gnu` out of box on alpine or even non-linux (surely without `+crt-static`)



❤️ 1



polarathene on Mar 11, 2024 · edited by polarathene

Edits ▼

Contributor

Author



`musl-gcc` is not needed when using `zigbuild`. btw, you can try `cargo zigbuild --target aarch64-unknown-linux-musl` to get an arm `openssl` static binary right away. (`musl-gcc` cannot do this)

That's a great benefit, thanks!

I can't seem to build for `-gnu` target from Alpine, but that's the case even with a basic "hello world" (cargo init) without the `openssl` crate 🤔

Why? `zigbuild` should work for `gnu` out of box on alpine or even non-linux (surely without `+crt-static`)

I was referring to without `zigbuild`, just noting that it didn't seem easy for Alpine to target `-gnu` with `cargo build`.

That along with the easy builds for different arch is great! ❤️

it has its [own approach](#) to deal with `glibc` symbols and does not rely on the system `glibc` at all

<https://github.com/ziglang/glibc-abi-tool#strategy>

*The only problem is when a function migrates from one library to another.
For example, in `glibc 2.32`, the function `pthread_sigmask` migrated from `libpthread` to `libc`, and the latest `abolist` files only show it in `libc`.
However, if a user targets `glibc 2.31`, Zig needs to know to put the symbol into `libpthread.so` and not `libc.so`.*

I detailed a reproduction case with my `glibc 2.32` concern with more information here: [#232 \(comment\)](#)

Disregarding static build:

- I noticed that `cargo build` will have a symbol version requirement (*I think that's the correct term?*) for glibc 2.32 for `pthread_getattr_np`. This was consistent on Fedora and Debian build hosts.
- `cargo zigbuild` continues to have the previous symbol version. Yet it is unclear if this change was missed by their process (*via the `glibc-abi-tool` you linked*), since for glibc 2.32 that symbol moved from `libpthread` to `libc`.
- It's apparently important for Rust to properly detect stack overflows?

So what happens in that scenario?

- Is it ok for the dynamic link build to not raise that min glibc requirement like it does with `cargo build` (*which makes the binary incompatible with earlier glibc versions*)?
- Why would the min version be raised for glibc 2.32 if `cargo zigbuild` can ignore that? Is it doing something differently that avoids that change?
- I'm curious if something like that may have contributed towards the segfault when trying to run a static `hello.c` `zig build` on the build host 🤔

Additional tip for dynamic glibc builds, how to check the minimum version linked:

```
# Parse the binary file for GLIBC symbols with specific format, then extract the section v
# followed by sorting major.minor fields numerically correctly, finally select the correct
$ readelf -W --version-info --dyn-syms target/x86_64-unknown-linux-gnu/release/bin-name-h
2.32
```



messense changed the title ``cargo zigbuild` doesn't support static builds with an explicit --target`` ``cargo zigbuild` doesn't support static glibc builds with an explicit --target`` on Mar 25, 2024

polarathene mentioned this on May 5, 2024

[Static build support \(openssl + cross-compile\) hickory-dns/hickory-dns#2207](#)

polarathene mentioned this on Aug 12, 2024

[Linking is not working correctly #254](#)

zachcoleman mentioned this on Aug 27, 2024

[Inclusion of toolchain forces `static=on` Go Targets to Dynamically Link](#)
[uber/hermetic_cc/toolchain#193](#)



polarathene on Dec 8, 2024

Contributor

Author



Update: Zig recently resolved [ziglang/zig#17268 \(comment\)](#) which should avoid one of the linking errors.

Zig still lacks support for glibc static builds that don't segfault but that may be resolved in future too. If they also resolve support for `-nostartfiles` linker flag, then Rust projects can leverage Zig with [eyra](#) as a better alternative to musl for static builds (*provided eyra has implemented anything needed for native deps to use in place of glibc*).



DrSensor last month



is it possible to target specific musl version? for example i want to link to musl 1.2.4 since the default rust musl is 1.2.3



Add a comment

H **B** *I* | | | @

Write

Preview

Use Markdown to format your comment



Paste, drop, or click to add files

Close issue



Comment

Metadata

Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

[Customize](#)

 [Subscribe](#)

You're not receiving notifications from this thread.

Participants

